SmartCourses

Dossier de lancement — M1 MIAGE Apprentissage — v0.1.0-SNAPSHOT

Yasmine HADDAG, Melissa AKLI, Kenza CHAHNAZ, Daniel BADOYAN, Hélios LOTT · 20 octobre 2026

01 Équipe, comptes développeurs et dépôt

MEMBRE	COMPTE GITHUB	EMAIL DES COMMITS (`git config user.email`)
Yasmine HADDAG	`@yasminehdg`	`email` (haddagyasmine9@gmail.com)
Melissa AKLI	`@Melissa-akli25`	`email` (melissa.akli2025@gmail.com)
Kenza CHAHNAZ	`@0Kenza0`	`email` (kenzachahnaz@outlook.fr)
Daniel BADOYAN	`@DanielBadoyan`	`email` (43003965@parisnanterre.fr)
Hélios LOTT	`@helioslt`	`email` (helioslott@gmail.com)

Chaque membre a vérifié que son email Git est associé à son compte GitHub, pour que ses commits lui soient bien attribués. **Dépôt** : https://github.com/Melissa-akli25/Projet-PRJ · **page de projet** : à compléter.

02 Projet et caractère innovant

Le problème

Faire ses courses en respectant son budget n'est pas toujours facile. Les prix varient d'un magasin à l'autre et il est parfois difficile de savoir où acheter ses produits au meilleur prix. De plus, choisir le magasin le moins cher n'est pas forcément la meilleure solution si celui-ci est trop éloigné et que le déplacement coûte plus cher que les économies réalisées.

SmartCourses a pour objectif d'aider les utilisateurs à mieux organiser leurs courses et à réduire leurs dépenses. L'application permet de créer une liste de courses, de rechercher des produits, de comparer les prix entre différents magasins et de constituer un panier adapté à son budget. Elle peut également proposer des produits alternatifs moins chers et prendre en compte la distance entre les magasins pour aider l'utilisateur à faire le meilleur choix.

Ce qui est innovant

- **L'optimisation plutôt que la simple comparaison.** SmartCourses ne se contente pas d'afficher les prix des produits. L'application cherche à proposer les solutions les plus avantageuses en tenant compte du budget de l'utilisateur et de ses besoins.
- **La prise en compte du coût du déplacement.** Le magasin qui propose les prix les plus bas n'est pas toujours le plus intéressant. SmartCourses compare les économies possibles avec la distance à parcourir afin d'aider l'utilisateur à déterminer si le déplacement vaut réellement le coup.
- **Des alternatives adaptées au budget.** Lorsqu'un produit est trop cher ou indisponible, l'application peut proposer des produits similaires moins chers. L'utilisateur peut ainsi réduire ses dépenses sans devoir renoncer systématiquement aux produits qu'il souhaite acheter.

Ce qui différencie SmartCourses

Les applications de courses peuvent permettre de rechercher des produits, de préparer des listes ou de consulter des prix. SmartCourses souhaite aller plus loin en combinant plusieurs critères pour aider l'utilisateur à prendre ses décisions.

L'objectif n'est donc pas seulement de trouver le produit le moins cher, mais de *déterminer la solution la plus avantageuse pour l'ensemble des courses, en tenant compte du budget, des prix et des déplacements*.

03 Scénarios d'usage

Scénario 1 : Création du profil utilisateur

En tant qu'utilisateur, je renseigne mon profil (foyer, budget, régime, placards, équipement), afin que le moteur d'optimisation génère un panier adapté à ma situation.

Préconditions

- L'utilisateur a téléchargé l'application et a créé un compte.
- L'utilisateur est connecté à son compte.

Postconditions

- Les paramètres de l'utilisateur sont sauvegardés dans la base de données.
- Le profil est prêt à être utilisé par le moteur d'optimisation pour générer un panier.

Scénario nominal

1. L'utilisateur accède à la page de configuration de son profil.
2. L'utilisateur saisit la composition de son foyer (nombre de personnes, adultes, enfants).
3. L'utilisateur configure son budget alimentaire de référence et son seuil de flexibilité.
4. L'utilisateur renseigne ses régimes alimentaires spécifiques et ses allergies.
5. L'utilisateur déclare les produits déjà présents dans ses placards et ceux proches de la péremption.
6. L'utilisateur indique son équipement de cuisine et le temps maximum souhaité pour la préparation des repas.
7. L'utilisateur valide le formulaire pour enregistrer son profil.
8. Le système enregistre les données et affiche un message de confirmation de création du profil.

Scénario alternatif

- **8.a** Le système détecte que des informations obligatoires sont manquantes ou incorrectes (ex. : budget négatif).
  - 8.a.1 Le système affiche un message d'erreur à côté des champs posant problème.
  - 8.a.2 Retour à l'étape 7 : l'utilisateur corrige sa saisie et revalide.

Scénario 2 : Optimisation du panier selon le budget

En tant qu'utilisateur, je donne ma liste de courses et mon budget maximum, afin d'obtenir un panier qui respecte ce budget grâce à des alternatives moins chères.

Préconditions

- L'utilisateur possède un compte SmartCourses et est connecté.
- Les produits et leurs prix sont disponibles dans l'application.
- L'utilisateur a défini un budget pour ses courses.

Scénario nominal

1. L'utilisateur crée une liste de courses et ajoute les produits qu'il souhaite acheter.
2. L'utilisateur indique le budget maximum qu'il souhaite respecter.
3. SmartCourses calcule le montant total estimé de la liste.
4. Si le montant dépasse le budget, l'application identifie les produits qui peuvent être remplacés par des alternatives moins chères.
5. SmartCourses propose plusieurs alternatives à l'utilisateur, en indiquant les économies possibles.
6. L'utilisateur consulte les propositions et choisit celles qui lui conviennent.
7. L'application met à jour le panier et affiche le nouveau montant estimé.

Scénario alternatif

- **4.a** Aucune alternative intéressante n'est disponible pour certains produits.
  - 4.a.1 SmartCourses indique les produits pour lesquels aucune alternative moins chère n'a été trouvée.
  - 4.a.2 L'utilisateur peut conserver ces produits ou modifier sa liste manuellement. # Plan des features

Une feature a un effet sur **la BD, le back, l'API REST et le front**. Chaque feature est portée par **une seule personne** et repose sur **un algorithme** identifié.

VERSION	YASMINE	MELISSA	KENZA	DANIEL	HÉLIOS
v0.1.0 24/11	À définir	À définir	À définir	À définir	À définir

VERSION	YASMINE	MELISSA	KENZA	DANIEL	HÉLIOS
v0.2.0 08/12	À définir	À définir	À définir	À définir	À définir
v1.0.0 19/01	À définir	À définir	À définir	À définir	À définir

04 Architecture

```
 Front  --HTTP/JSON⟶  API REST  ⟶  Back (cas d'utilisation)  --JPA⟶  BD
(React)              (Spring Web)       (Java, Spring Boot)            (MySQL)
```

Le front transmet les demandes à l'API REST. Le back exécute les règles métier et les algorithmes (optimisation du panier, équilibrage nutritionnel, substitution de produits), accède à la base de données, puis renvoie les résultats au front au format JSON. L'API REST est intégrée à l'application Spring Boot.

COUCHE	CHOIX	REMARQUES
Front	React, JavaScript, HTML/CSS	saisie du budget et des préférences alimentaires, consultation des paniers proposés ; client de l'API REST uniquement ; dossier `front/`
API REST	Spring Web (Spring Boot), OpenAPI / Swagger UI	échanges HTTP et JSON ; versionnée sous `/api/v1`
Back	Java, Spring Boot, Spring Data JPA, Hibernate, Bean Validation	règles métier et algorithmes ; **algorithmes en Java pur, indépendants de Spring**
BD	MySQL (H2 pour les tests)	produits, informations nutritionnelles, prix par magasin, paniers ; schéma versionné par **Flyway**
Qualité	JUnit 5, JaCoCo, SonarCloud	intégration continue GitHub Actions
Conteneurs	Docker Compose	BD dès v0.1.0, application en v0.2.0

Qualité du code. Au-delà de l'architecture :

- **Clean Code** : noms métier explicites, fonctions courtes, pas de duplication, commentaires qui expliquent le *pourquoi* ;
- **SOLID**, objets valeur immuables et invariants vérifiés à la construction ;
- **design patterns** quand ils servent, nommés dans les fiches de features ;
- conventions **REST** et règles de sécurité **OWASP** ;
- *quality gate* SonarCloud au vert.

05 Démarche pour chaque feature

Les étapes s'enchaînent **dans cet ordre, sans en sauter** : on ne commence une étape que lorsque la précédente est terminée.

ÉTAPE	CE QU'ON FAIT	TERMINÉ QUAND
1. Idéation	Besoin et scénario ; **algorithme retenu**, complexité, alternatives ; impact sur le schéma et l'API ; critères d'acceptation. Issue + fiche `doc/features/` .	La fiche est validée par un autre membre.
2. Back	Migration Flyway ; **domaine** en Java pur (modèle, règles, algorithme) ; **application** (ports, service) ; adaptateur de persistance.	Le code compile et ArchUnit passe.
3. Tests	Unitaires de l'algorithme (cas limites, invariants, propriétés) ; services avec faux adaptateurs ; intégration de la persistance.	`./gradlew build` passe ; chaque critère d'acceptation est couvert.
4. API REST	Endpoints `/api/v1` , DTO validés, erreurs `problem+json` , OpenAPI ; tests de l'API.	Visible dans Swagger UI ; tests de l'API verts.
5. Front	Écrans qui consomment **uniquement** l'API.	Scénario jouable de bout en bout ; PR relue ; fiche complétée avec son araignée.

06 Protocole d’usage de l’IA

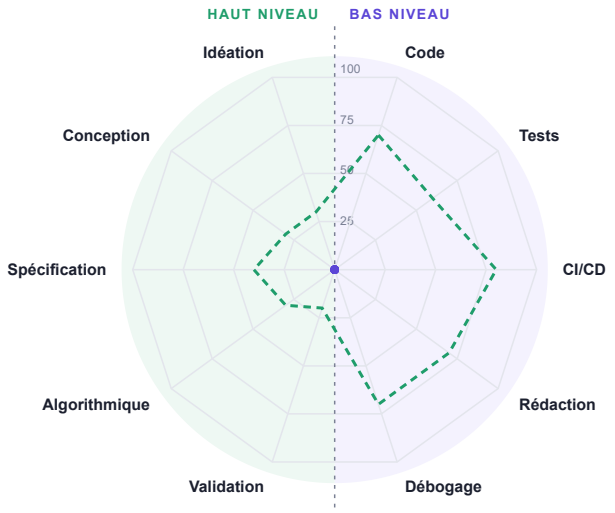
Principes fondamentaux à appliquer pendant tout le long de ce projet :

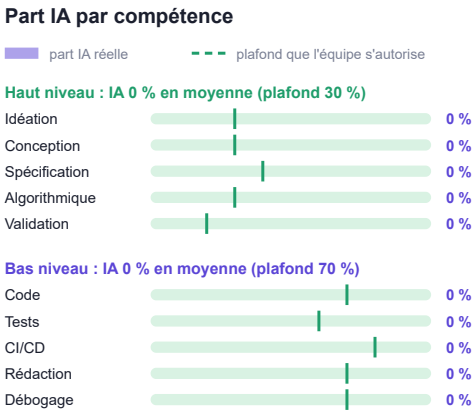
- Vérifier au maximum les résultats qui vont nous être donnés par les intelligences artificielles que nous serons amenés à utiliser
- Chaque prompt devra être réfléchi et notre utilisation de l’IA devra être intelligemment pilotée afin de ne pas perdre le fil de notre travail et des algorithmes/codes que nous serons amenés à écrire.
- Interdiction d’inclure du code qu’on ne comprend pas et de confier à l’IA des données personnelles.

ACTIVITÉ	CE QUE FAIT L’IA	CE QUE FAIT L’HUMAIN
Recherche du sujet et conception des objectifs du projet	Aide l’humain dans son choix des fonctionnalités à inclure notamment par rapport à la pertinence vis-à-vis des consignes et pour éviter de faire un projet type « interface d’une simple base de données »	Trouver le sujet du projet et réfléchir aux fonctionnalités techniques et algorithmiques qui pourraient inclues
Algorithmes	Vérifie le bon fonctionnement du code, elle peut également l’annoter et le rendre plus agréable à la lecture en appliquant les bonnes pratiques d’écriture	Rassemble les attentes de rendu des algorithmes puis les conçoit et les écrit en prenant en compte la complexité
Code technique (API, configuration, front)	Peut générer certaines parties notamment les fonctions les plus complexes	Relie chaque ligne générée et compile avant d’adapter le code en fonction des besoins pour répondre au sujet
Tests	Suggère les cas de test les plus courants pour ce type de projet	Définit les propriétés, vérifie les tests effectués et adapte les propositions de de l’IA en fonction de l’avancée du projet
Débogage et revue	Propose des diagnostics et résout les éventuels bug persistants	Analyse les erreurs et corrige en conséquence

Afin d’assurer une traçabilité de l’usage de l’IA dans notre projet, les codes en èrement générés seront annotés en conséquence avec la précision des technologies u lisées et le niveau d’u lisa on (l’écriture complète étant le niveau le plus élevé et la simple demande de recommanda ons ou de perspec ves d’améliora on étant plus faible).

07 Annexe : part IA / part humaine





Valeurs dans `doc/part-ia.json`, régénérées à chaque compilation.